When Distance Shapes the Story: Clustering with K-means

Liran Ma, Zhipeng Cai

Contents

1 Introduction

Lina is doing an internship in NYC and living with two roommates. To save time on weeknights, they set a simple rule: whoever gets off work first picks dinner, and everyone else just follows.

Week one is great. But in week two, things go sideways. After trying a random spot near the apartment, everyone feels off the next day. One roommate says: "Why do we always end up picking the risky places?"

Lina tries the usual approach: scrolling through reviews. It doesn't help. The same restaurant can have five-star praise and one-star horror stories. It's noisy, subjective, and not very useful when you're hungry at 7 p.m.

Then she remembers something more objective: NYC publishes official *Restaurant Inspections* data. It includes inspection dates (`INSPECTION DATE`), inspection scores (`SCORE`), locations (`Latitude / Longitude`), cuisine type (`CUISINE DESCRIPTION`), borough (`BORO`), and the restaurant name (`DBA`).

So Lina decides to do the most "data-person" thing possible: build a simple "dinner risk map." The idea is to use clustering to group restaurants that look similar, then see what patterns show up.

Your job in this lab

In this manual, you will help Lina build an interactive clustering panel step by step:

- **Step 1 (Geo only):** cluster restaurants using `Latitude/Longitude`.
- **Step 2 (+ Score):** add `SCORE` and see how the clusters change.
- **Step 3 (+ Cuisine):** include cuisine one-hot features and adjust the cuisine weight α.

By the end, you should be able to produce:

- a NYC map colored by clusters, and
- a short, readable table describing what each cluster represents.

2 Dataset Overview: NYC Restaurant Inspections

Goal. In an unsupervised problem, there is no correct label to learn from. The "answer" depends on how we represent each restaurant (which features we choose) and how we measure similarity.

2.1 Dataset overview

We use the NYC Open Data *Restaurant Inspections* dataset. One row in the raw file is one *inspection record* (not one restaurant).

2.2 Common fields

The raw dataset contains many columns. Here are the key ones we will use in this lab:

- **`CAMIS`:** a unique ID for a restaurant (used to group records that belong to the same place).
- **`DBA`:** the restaurant name as it appears in the inspection record.
- **`BORO`:** the NYC borough (e.g., Manhattan, Brooklyn, Queens).
- **`INSPECTION DATE`:** the date when the inspection happened.
- **`SCORE`:** inspection score (higher typically indicates more violations / worse inspection outcome).
- **`Latitude/Longitude`:** the restaurant's location used for mapping and geographic clustering.
- **`CUISINE DESCRIPTION`:** a category label for cuisine type (e.g., Pizza, Chinese, Mexican).

2.3 Why do we need restaurant-level data?

A single restaurant can appear many times because it can be inspected multiple times. If we cluster the raw rows, restaurants with frequent inspections would get extra weight.

In this manual, we convert the data into a restaurant-level table by keeping **one row per restaurant**: we take the **most recent inspection record** for each `CAMIS`. This choice is simple, easy to explain, and easy to reproduce.

Think

1. What differences do you expect between using the *latest* score vs. the *average* score for each restaurant?
2. Suppose a restaurant is inspected many times in one year. If we cluster inspection-level rows, how could that distort the clustering results? Why?

3 Core Ideas: What Is K-means Doing?

Goal. Build a solid, technical intuition for K-means with minimal math: the objective it optimizes, the algorithmic loop it runs, and the specific ways it can mislead you in practice.

3.1 What is clustering?

Clustering is **unsupervised learning**: there is no outcome y to predict. Instead, we choose a feature representation $x_i \in R^p$ for each restaurant and ask the algorithm to partition points into k groups.

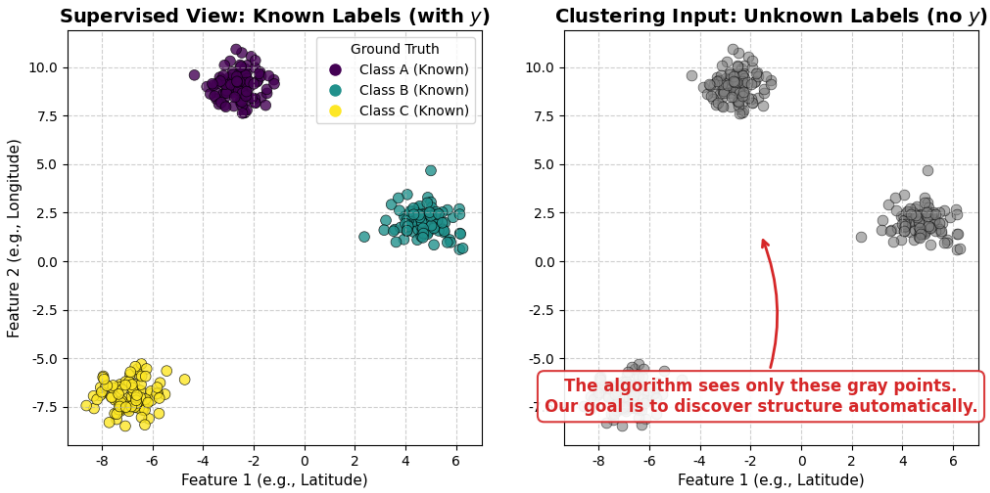


Figure 1: **Supervised vs. Unsupervised intuition.**

Figure 1 shows the key mindset shift: in clustering, we do not have "correct answers" during training. We only have the input features, so what we discover depends entirely on how we represent each restaurant as x_i and how we define similarity (distance).

In this lab, the same restaurant can be represented in different feature spaces depending on the step:

- **Step 1:** $x_i = (\texttt{Lat}, \texttt{Lon})$
- **Step 2:** $x_i = (\texttt{Lat}, \texttt{Lon}, \texttt{SCORE})$
- **Step 3:** $x_i = (\texttt{Lat}, \texttt{Lon}, \texttt{SCORE}, \text{one-hot cuisine})$

So the question is never just "What are the clusters?" It is "What clusters do we get under *this* definition of a restaurant and *this* definition of similarity?"

3.2 The objective: what K-means is optimizing

K-means assumes each cluster can be summarized by a single center (a **centroid**). Formally, it tries to find:

- an assignment of each point to a cluster, and
- k centroids $\mu_1, \ldots, \mu_k$,

that minimize the **within-cluster sum of squared distances** (WCSS), also called **inertia**:

$$\sum_{j=1}^{k} \sum_{i \in C_j} \|x_i - \mu_j\|_2^2.$$

Interpretation: we want points to be close to their assigned centroid; we pay a quadratic penalty for being far away.

Two immediate implications:

- K-means prefers **compact, roughly spherical** clusters in the feature space.
- Outliers and long "tails" can matter a lot because of the squared distance.

3.3 The algorithm loop: assignment → update → repeat

K-means is an iterative algorithm (Lloyd's algorithm). It alternates between two steps:

(1) Assignment step. Given current centroids $\mu_1, \ldots, \mu_k$, assign each point to the nearest centroid:

$$\text{cluster}(x_i) = \arg \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|_2.$$

In code terms: "compute distances to all centroids, take the smallest."

(2) Update step. Given current cluster assignments, update each centroid to be the **mean** of the points assigned to that cluster:

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i.$$

Why the mean? Because the mean is the minimizer of squared error within a cluster.

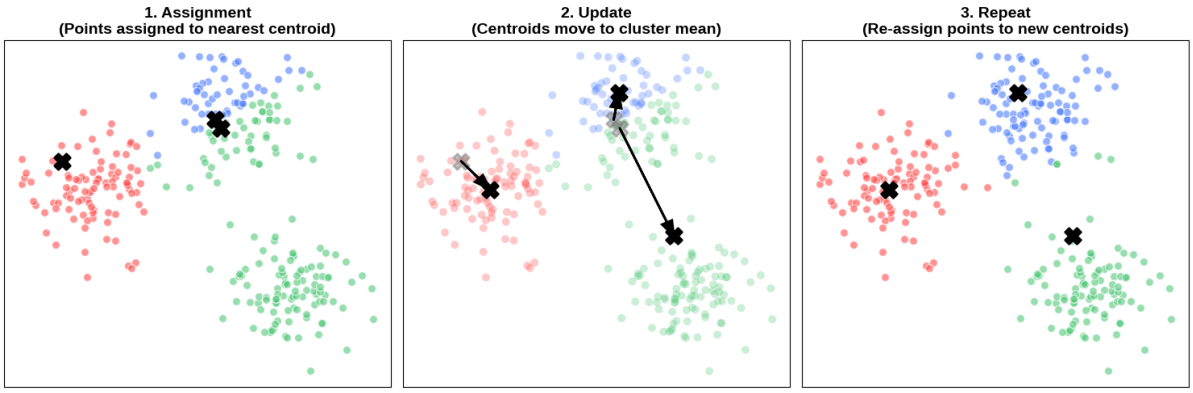


Figure 2: The K-means loop in two steps.

Read Figure 2 as a cycle: **assign** points to the nearest centroid, then **update** each centroid to the mean of its assigned points. Each round can only decrease (or keep) the within-cluster sum of squares (WCSS), so the algorithm moves toward a stable configuration.

This loop continues until convergence, typically when:

- assignments stop changing, or
- the decrease in objective (WCSS) becomes tiny, or
- a max number of iterations is reached.

3.4 Initialization and local minima (why results can change)

K-means is **not** guaranteed to find the global optimum. It converges to a **local minimum** that depends on how centroids are initialized.

In practice, most implementations use **k-means++** initialization to spread out starting centers, and run multiple random restarts (`n_init`) to reduce the chance of a bad local solution. That is why two runs can produce different clusters if you do not fix a random seed.

In this lab code, we set `random_state=42` for reproducibility.

3.5 Distance and scale: K-means' worldview

K-means uses **Euclidean distance**. That means "similarity" is determined entirely by geometry in feature space.

Scale matters. If one feature has a larger numeric range, it can dominate the distance. For example, raw `SCORE` can vary much more than `Lat/Lon` in their units, so without scaling, clusters may be driven mostly by score (or mostly by geography), depending on magnitudes.

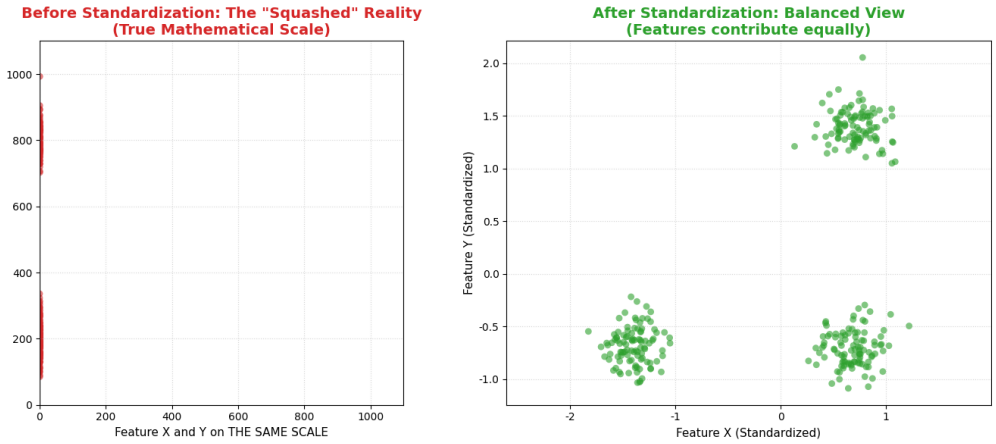


Figure 3: Standardization changes the geometry that K-means uses.

Figure 3 shows the core problem: **K-means only sees distances.** Before standardization (left), one feature can have a much larger numeric range, so it dominates Euclidean distance. The result is a "squashed" view where variation in the smaller-scale feature barely affects distance. After standardization (right), each feature has mean 0 and standard deviation 1, so differences along each axis contribute more evenly.

That is why we often standardize features using `StandardScaler`:

$$z = \frac{x - \mu}{\sigma},$$

so each feature has mean 0 and standard deviation 1. After scaling, each dimension contributes more comparably to Euclidean distance.

Intentional weighting. In Step 3, we add cuisine one-hot features and optionally multiply them

by a weight α. Conceptually, this is the same as changing the metric: you are telling K-means "differences in cuisine should count α times as much." When $\alpha = 0$, cuisine is ignored; when α increases, cuisine increasingly drives the geometry and therefore the clusters.

3.6 How do we pick k (number of clusters)?

There is no single "correct" k. A useful k is one that balances:

- **separation:** clusters are meaningfully different,
- **stability:** results do not flip wildly with small tweaks, and
- **interpretability:** clusters are easy to explain in plain language.

Silhouette score (intuition, but with the right caveat). For each point, silhouette compares "how close am I to my own cluster?" against "how close am I to the nearest *other* cluster?" The average over all points lands between -1 and 1; higher means points fit their assigned cluster better *in the current feature space.*

However, silhouette is not a guarantee of usefulness:

- **It only knows your feature space.** If the feature space is poorly chosen (for example, dominated by one-hot cuisine columns), you can get a high silhouette for clusters that are crisp geometrically but meaningless for your real question.
- **It can reward unhelpful structure.** Splitting off a handful of outliers into a tiny cluster, or carving one cuisine into its own group, can raise the score while making the result harder to explain. A "better" number does not mean a better dinner-risk map.
- **It is not monotone in k.** Unlike WCSS (which always goes down as k grows), silhouette usually peaks at some moderate k and then declines as clusters start crowding each other. So you cannot game it just by adding clusters, but you also should not treat its peak as the "true" k.

In this lab, we treat silhouette as a **diagnostic, not a final answer**: the final check is whether the clusters form a story you can explain and defend.

Think

1. If you multiply one feature by 10 (for example, `SCORE`), what do you expect K-means to do differently? Why?
2. Does a higher silhouette score always mean the clustering is more meaningful for the real task? Explain your reasoning in one or two sentences.

4 From Raw Data to Modeling Data: Cleaning and Building a Restaurant Table

This part decides whether your clusters are believable. Before we run K-means, we need a dataset that is clean, consistent, and (most importantly) **one row per restaurant**.

How this section maps to your notebook

In the notebook, the work here happens across a few cells. Run them in order:

- **Cell 1:** Import packages (quick test that your environment is ready).
- **Cell 2:** Download the raw NYC inspections CSV and do a quick sanity check (`shape` + `head`).
- **Cell 3:** Convert the inspection-level table into a restaurant-level table (`rest_latest`).
- **Cell 4:** Build features and launch the interactive K-means panel (we will focus on that in the next section).

4.1 What the raw file actually represents (inspection-level)

NYC Restaurant Inspections is an **inspection-level** dataset: one row is one inspection event. So the same restaurant (same `CAMIS`) can show up multiple times across different dates, with different scores and grades. If we cluster the raw rows directly, restaurants that were inspected more often would get extra weight by accident.

What we want for clustering is a **restaurant-level** dataset: one row per restaurant, so each restaurant counts once.

4.2 Cleaning: small fixes that prevent big problems

Public datasets are full of predictable "messy" patterns. Your notebook handles the key ones so the map and the distance calculations behave normally:

- **Dates:** make sure `INSPECTION DATE` is treated as a real date. Why it matters: if dates are broken, "latest inspection" becomes meaningless.
- **Coordinates:** remove missing coordinates and drop obvious junk locations like zeros. Why it matters: bad coordinates can stack many points at the same place and create a fake "hotspot" on the map.
- **Scores:** ensure `SCORE` is numeric and drop rows where it is missing. Important: in this dataset, a score of 0 is a real (and good!) result — it means the inspection found zero violation points. Do not treat 0 as a placeholder and delete it. Missing scores show up as blank/NaN, not as zeros. Why it matters: if we accidentally drop the 0-score rows, we silently remove the

cleanest restaurants from the dataset, and every cluster's `mean_score` will look worse than reality — exactly the wrong thing for a dinner-risk map.

- **Cuisine labels:** fill missing `CUISINE DESCRIPTION` with `Unknown`. Why it matters: we keep the restaurant in the dataset instead of dropping it just because the label is missing.

4.3 From inspection-level to restaurant-level (Cell 3)

After cleaning, we still have multiple rows per restaurant. In **Cell 3**, you build `rest_latest` by:

- sorting rows by `CAMIS` and `INSPECTION DATE`, then
- keeping the **most recent** row for each `CAMIS`.

This produces the modeling table we will use for clustering: each row represents one restaurant with a location (`Latitude/Longitude`) and key context fields (like `SCORE` and `CUISINE DESCRIPTION`).

Your job in this lab

What you should see after Cell 3
You should have a dataframe called `rest_latest` where:

- each `CAMIS` appears only once (no duplicates),
- `SCORE` is mostly non-missing (depending on your cleaning choices),
- the preview shows fields like `DBA`, `BORO`, `CUISINE DESCRIPTION`, `INSPECTION DATE`, `SCORE`, and coordinates.

If any of these checks fail, fix it now—everything in the interactive panel depends on this table.

Reality check

A cluster that looks "too perfect" can come from placeholder or coded values (for example, default dates like 1/1/1900, which NYC uses for "new establishment, not yet inspected"). And be careful in the other direction too: `SCORE`=0 is a legitimate "zero violations" result, not a placeholder: deleting it quietly biases your scores upward. K-means is doing exactly what you asked: so cleaning is part of the model, not an optional extra.

Where we go next (Cell 4)

Now that we have `rest_latest`, we are ready to build feature matrices and run K-means. In the next section, we will use the interactive panel to compare: **(1) Geo only**, **(2) Geo + Score**, and **(3) Geo + Score + Cuisine**.

5 The Three-Step Experiment

This is the core storyline of the lab. The idea is simple: each step follows the same rhythm — **predict → run → read the output → say what changed (and why)**.

Rule of thumb: do not chase the highest silhouette score. In this lab, we care more about whether the clusters are *interpretable* and whether the changes across steps make sense.

5.1 Step 1: Geo only (Latitude/Longitude)

Goal. Use location only to help you see what clustering looks like on a map. At this point, K-means is basically trying to **divide NYC into regions**.

What you are using

In the panel, set:

- `Step = 1 (Geo only)`
- `Scaling = True` (keep it on for consistency across steps)
- Try a few values of `k` (for example: `k=3`, `k=4`, `k=5`)

What you should notice

When you change `k`, you are not "improving truth" — you are changing the **resolution** of the map.

- **Map:** Do the colored regions look like reasonable geographic chunks?
- **Summary table:** Check `mean_lat` and `mean_lon`. If Step 1 is behaving as expected, different clusters should have clearly different average locations.
- **One more check:** look at `pct` (cluster share). Are you getting one giant cluster and a few tiny ones, or a more balanced split?

A common surprise: with Geo-only, clusters often align with obvious geography (borough shapes, dense areas, islands). That's not "deep insight" yet — it's a sanity check that K-means is behaving.

What to write down (one sentence each)

After you try `k=3`, `k=4`, and `k=5`, write:

- Which `k` gives the cleanest-looking regions on the map?
- Which `k` is easiest to explain to a friend who has never seen clustering?

Think

1. When `k` goes from 3 to 8, does the map become clearer or more fragmented? Why might that happen?
2. NYC has natural structure (water, bridges, dense downtown areas). How do you expect that structure to show up in Geo-only clustering?

5.2 Step 2: Geo + Score (and why scaling suddenly matters)

Goal. In Step 1, distance meant "how far apart on the map." In Step 2, distance becomes "map + inspection score." That one extra column changes what it means for two restaurants to be *similar*.

What you are using

In the panel, set:

- `Step = 2 (Geo + Score)`
- Fix `k` (use the same `k` you liked in Step 1; `k=5` is a good default)
- Run **twice**:
  - **Run A:** `Scaling = True`
  - **Run B:** `Scaling = False`

Why we force the scaling comparison: K-means only knows Euclidean distance. If one feature has a larger numeric scale, it can quietly dominate the distance calculation and take over the clustering.

How to read the output (map + summary table)

You now have `mean_score` in the summary table, plus `pct` and the `examples`. Use them like a detective:

- **Map:** Do clusters still look like geographic regions, or do you see mixing within the same area?
- **Summary table:**
  - `mean_lat / mean_lon`: where the cluster "lives."
  - `mean_score`: whether the cluster tends to be "cleaner" or "riskier" (higher usually means more violations).
  - `pct`: how big the cluster is (one giant + many tiny clusters is a warning sign).
  - `examples`: quick reality check—does the cluster have a story you can explain?

What should change when you add SCORE?

Adding `SCORE` gives K-means a new reason to separate restaurants: two places can be next door, but if their inspection outcomes are very different, they may land in different clusters.

A good Step 2 result often looks like:

- clusters still roughly follow geography,
- but within a region, you start seeing separation by score (one cluster with higher `mean_score`, another with lower `mean_score`).

Quick checkpoint (no math): If two restaurants are across the street from each other but end up in different clusters in Step 2, ask: *"Is their `SCORE` very different?"* If yes, Step 2 is doing exactly what you asked.

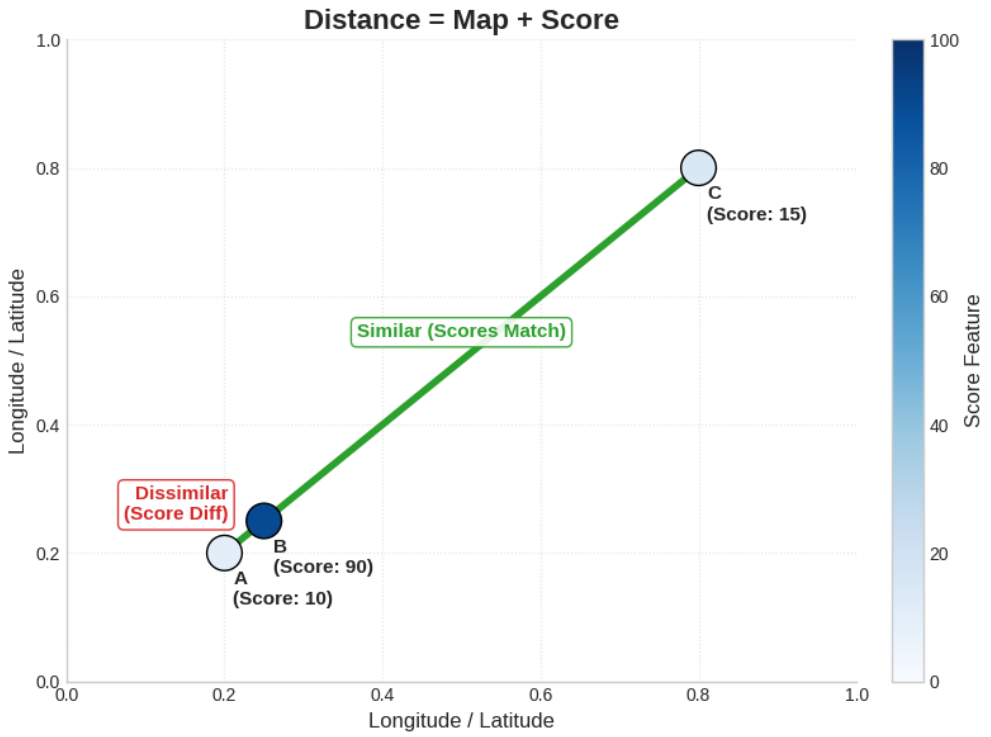
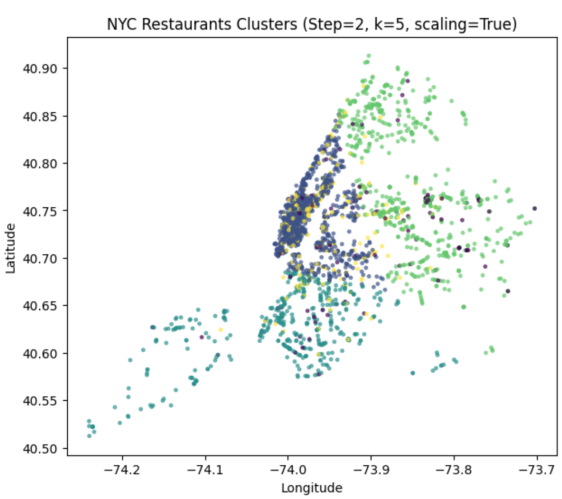


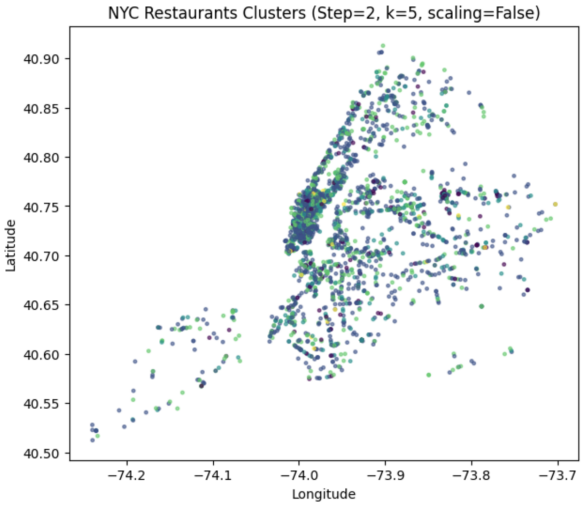
Figure 4: Distance = Map + Score

Scaling ON vs OFF (the whole point of Step 2)

These two plots are intentionally shown **together**—they are a controlled comparison. You are not trying to decide which picture is "prettier". You are learning what scaling does to *distance*.



(a) `Scaling=True`: Geo and Score contribute more fairly.



(b) `Scaling=False`: one feature can dominate distance and distort clusters.

Figure 5: Step 2 is a "scale experiment": same data, same `k`, but different distance geometry.

Think

1. In your own words: what changed more—the geography of clusters, or the score pattern inside clusters?
2. If your goal is "mostly geographic clusters, with score as a smaller influence," name **two** practical ways to do that in this lab (hint: scaling/weighting/feature choice).

5.3 Step 3: + Cuisine one-hot (optional)

Goal. Now we add a high-dimensional categorical signal: `CUISINE DESCRIPTION`. This is where clustering starts to feel *less like geography* and more like *"restaurant identity."*

What you are using

In the panel, set:

- `Step = 3 (+ Cuisine one-hot)`
- `Scaling = True` (keep it on—otherwise cuisine features can behave unpredictably)
- Choose a value of `k` (start with the same `k` you used in Step 2)
- Tune two knobs:
  - `Top N cuisines`: how many cuisine types get their own one-hot columns
  - `Cuisine weight` $\alpha \in [0, 1]$: how loud cuisine is in the distance calculation

Two knobs, two meanings. `Top N` controls *how many cuisine dimensions you add.* α controls *how much those dimensions matter.* Top-N is "how many speakers"; α is "the volume."

What changes (and why)

Cuisine is not one number like score—it becomes **many** binary columns (one-hot). That means you are adding a **high-dimensional block of features**. Even though each column is only 0/1, there are many of them, so cuisine can heavily reshape distance.

- **When $\alpha = 0$:** Step 3 collapses back to Step 2. You are saying: "ignore cuisine; only use Geo + Score."
- **When α increases:** cuisine becomes a stronger voice. Restaurants sharing cuisine become "closer" (even if they are far apart geographically).

Two signature patterns to watch for

- **Pure cuisine clusters (e.g., `Pizza(100%)`).** If the summary shows `Pizza(100%)`, that cluster is being formed almost entirely by cuisine. One-hot features can dominate similarity when we let them.
- **The "confetti map" (mixed colors everywhere).** If the map stops looking like regions and turns into confetti, it means clustering is no longer mainly driven by location. *you turned the dial from "where you are" to "what you are."*

How to run Step 3 like an experiment (recommended)

Keep `k` fixed. Keep scaling on. Then try:

$$\alpha = 0,\ 0.3,\ 0.6,\ 1.0$$

Your job is not to find the "best" α. Your job is to **see the trade-off**: as cuisine gets louder, geography gets quieter.

How to read the outputs (map + table)

Use the two outputs together:

- **Map:** Do clusters still look like areas, or do they scatter across NYC?
- **Summary table:**
  - `top_cuisines`: does each cluster have a clear theme?
  - `examples`: do the example restaurants match that theme?

– `pct`: did you accidentally create tiny niche clusters?

Think

1. Why can one-hot cuisine features have strong influence on K-means distance, even though each one is just 0/1?
2. If silhouette keeps improving as `k` increases, which `k` would you trust more here—the largest one or the easiest one to explain? Why?
3. When "model score" and "business interpretability" conflict, which one should Lina choose for a dinner-risk map, and why?

Wrap-up: Answer Lina's Question

Remember why we started: Lina and her roommates want to stop picking risky dinner spots. Clustering was never the goal, but the tool. So, textbfone configuration from your experiments and commit to it. A configuration means a specific choice of Step, k, and scaling (plus Top N and α if you used Step 3). Using its map and summary table, write a short "memo to the roommates" (5–8 sentences) that answers:

1. **Which cluster(s) would you avoid for weeknight dinners, and why?** Cite at least two columns from the summary table (e.g., `mean_score`, `pct`, `examples`) as evidence — "the colors looked different" is not evidence.
2. **Which cluster looks like the best default choice near where Lina lives?** (Pick any neighborhood and commit to it.)
3. **One honest limitation.** Name one reason your map could mislead the roommates (think: the latest-score-only choice from Section 4, what `SCORE` does and doesn't measure, or what your α setting silenced).